Architecture.md v1.1

Chapter 4 — Communication Protocol & Internal Contracts

This chapter defines how every component inside AI OS communicates.

No component may invent its own communication method.

All modules shall follow the protocols defined in this chapter.

4.1 Communication Philosophy

AI OS uses **contract-based communication**.

Components never communicate by assuming another component's implementation.

Instead, they exchange standardized requests and responses.

Benefits:

- Loose coupling
- Easier testing
- Easier replacement
- Better debugging
- Cleaner architecture

4.2 Communication Hierarchy

Every request follows this hierarchy.

```
User
   │
   ▼
Router
   │
   ▼
Jarvis / Friday
```

```
   |
   ▼
Agent Manager
   |
   ▼
Worker Agent
   |
   ▼
Memory Service (if needed)
```

Responses travel back through the exact same path.

No shortcuts are allowed.

4.3 Standard Request Contract

Every internal request must contain the same structure.

Required fields:

- Request ID
- Timestamp
- Source Component
- Destination Component
- Task Name
- Task Description
- Context
- Priority
- Permission Level

Example:

Request ID: `REQ-000124`

Source: `Jarvis`

Destination: `Analytics Agent`

Task: `Analyze Instagram Growth`

Context: Previous conversation, user preferences, project data.

4.4 Standard Response Contract

Every component returns a standardized response.

Required fields:

- Request ID
- Agent Name
- Status
- Result
- Error (if any)
- Execution Time
- Next Recommendation (optional)

Status values:

- SUCCESS
- FAILED
- WAITING_FOR_APPROVAL
- PARTIAL_SUCCESS
- CANCELLED

No custom status values are permitted.

4.5 Request Lifecycle

Every request follows these stages:

1. Created
2. Routed
3. Assigned
4. Executing
5. Completed
6. Logged
7. Returned

Every stage generates an event for monitoring.

4.6 Agent Pipeline

The Agent Manager may build execution pipelines.

Example:

```
Research Agent
        │
        ▼
Analytics Agent
        │
        ▼
Content Agent
```

Worker agents never communicate directly.

Instead:

Research Agent

↓

Agent Manager

↓

Analytics Agent

↓

Agent Manager

↓

Content Agent

The Agent Manager transfers intermediate context.

4.7 Context Passing Rules

Context passed between agents must include only relevant information.

Example:

Analytics Agent should receive:

- Instagram metrics
- Target account

- Time range

It should NOT receive:

- Business invoices
- Calendar data
- Unrelated memories

Context should always be minimal.

4.8 Memory Communication Rules

Worker agents cannot access the database.

Instead:

Worker Agent

↓

Memory Service

↓

Database

Only the Memory Service understands persistence.

4.9 Error Communication

Errors must never terminate communication unexpectedly.

Every failure returns a structured response.

Example:

Status:

FAILED

Reason:

API unavailable.

Suggested Action:

Retry later.

No raw exceptions should reach Jarvis or Friday.

4.10 Approval Workflow

Sensitive actions require user approval.

Workflow:

```
User Request
      │
      ▼
Worker Agent
      │
      ▼
Approval Required
      │
      ▼
Agent Manager
      │
      ▼
Jarvis / Friday
      │
      ▼
User
```

Possible outcomes:

Approve

↓

Continue execution

Reject

↓

Cancel task

Timeout

↓

Return CANCELLED

4.11 Event Communication

Every important action creates an event.

Examples:

- Request Received
- Agent Started
- Agent Finished
- Memory Updated
- Approval Requested
- Error Occurred
- Task Cancelled

Events are sent to the Event Service.

Events are never sent directly between worker agents.

4.12 Communication Restrictions

Forbidden:

❌ Worker Agent → Worker Agent

❌ Worker Agent → User

❌ Worker Agent → Database

❌ Router → Worker Agent

❌ Jarvis → Database

❌ Friday → Database

Allowed:

✅ Router → Jarvis

✅ Router → Friday

✅ Jarvis → Agent Manager

✅ Friday → Agent Manager

✅ Agent Manager → Worker Agent

✅ Worker Agent → Memory Service

✅ Memory Service → Database

4.13 Timeouts

Every request has a timeout.

If exceeded:

1. Agent stops execution.
2. Partial work is discarded unless explicitly saved.
3. Error is logged.
4. Status becomes FAILED.
5. Agent Manager decides whether to retry or continue.

No request should wait indefinitely.

4.14 Retry Policy

Only the Agent Manager may retry tasks.

Worker agents never retry themselves.

Retry policy (v0.1):

- Maximum retries: 2
- Delay between retries: configurable
- Retry only for recoverable errors

Examples:

Retry:

- Temporary API failure
- Network timeout

Do not retry:

- Invalid user input
- Permission denied
- Unsupported task

4.15 Communication Principles

Every communication inside AI OS must satisfy:

- Predictable
- Traceable
- Logged
- Replaceable
- Secure
- Minimal
- Standardized

No hidden communication channels are allowed.

4.16 Design Rule

If two components need to exchange information, they must do so through the architecture defined in this chapter.

No future feature may bypass these communication contracts without an approved architecture revision.

End of Chapter 4

Next Chapter: Chapter 5 — Shared Memory Architecture